



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

PARSER AND SYNTAX DIRECTED TRANSLATION

Course: Compiladores

Group: 05

Professor: M.C. René Adrian Dávila Pérez

Student(s):

Ambriz Cano Diego Emilio

Casillas Juárez Camila

Ferrer Cordero José Manuel

González Casimiro Juan Carlos

Vidrio López Mario Alexis

Table of Contents

Introduction **3**

Top-Down Parsing **3**

Lexer tokens **4**

Parser and SDT **6**
Verify. Leftmost derivation 9

Verification of the Grammar for Top-Down Parsing **9**
Unambiguous 9
Without left recursion 10
Left factoring 10
Condition (LL1) 10

First **11**

Follow **12**

First and Follow Summary **14**

Parse table **14**
Parse table complete 16

LL(1) Tests **16**

Designing SDT and rules **18**
Attributes 19
Type System 19
Symbol Table and Addresses 20
Semantic rule for each production 20

Evaluating the SDT **22**
Input: `int x = 10;` 22
Correct parsing, incorrect SDT 23

Test cases **24**
P1: valid input 24
P4: parsing error 24
S1: implicit conversion allowed 25
S3: undeclared variable 25
S7: non-numeric operand 25

Conclusions **25**

References **26**

Introduction

In the compilation process, the **Parser** (or syntactic analyzer) plays a fundamental role. While the previous phase—the **Lexical Analyzer (Lexer)**—converts the raw input stream of characters into a sequence of classified tokens, the Parser receives these tokens to verify their grammatical structure. Its primary function is to build a hierarchical data structure known as a **Parse Tree**.

Building upon the previous work on the *Lexical Analyzer*, which generated the token sequence required for syntactic analysis, this document details the complete design and implementation of a **non-recursive Top-Down LL**(1) **Parser**. Although various strategies exist for parser construction, this work focuses strictly on the aforementioned deterministic approach.

Top-Down Parsing

Top-Down Parsing (or Predictive Parsing) is a parsing method in compiler theory that attempts to construct the syntax tree starting from the initial symbol (or root) of the formal grammar downwards, successively applying its production rules to the non-terminal symbols until generating an exact string of terminal symbols matching the source code input.

In this project, a non-recursive LL(1) predictive Top-Down parser was chosen. Its classification is Top-Down, non-backtracking, and predictive. The acronym stands for: reads the input from Left to right (L), constructs a Leftmost derivation (L), and decides using 1 symbol of lookahead.

The following comparative table presents some of the arguments supporting the construction of the Parser using the LL(1) Top-Down strategy.

Criterion	Top-Down LL(1)	Bottom-Up LR(0), SLR(1), LALR(1), CLR(1)
Tree construction	From the root to the leaves	From the leaves to the root (shift-reduce)
Grammars accepted	LL(1) grammars, verified through their selection sets and predictive table.	More grammars, including left recursion.
Compatibility with the grammar	Yes: it is already LL(1) (Table 8)	Yes, but their main advantage, accepting a wider class of grammars (including left recursion), is not needed here because the grammar is already LL(1).
Table construction	Directly from FIRST and FOLLOW: 12×17	Requires the collection of item sets and the LR automaton
Trace	Stack, input, and action.	Stack of states and symbols, with shifts and reductions

Table 1. Comparison between Top-Down LL(1) and Bottom-Up parsers.

LL(1) was chosen because the grammar allows decision-making with a single lookahead token and because the table and execution trace facilitate verifying the procedure. Inherited attributes do not dictate the choice of a top-down parser; in this project, they are evaluated on the complete parse tree after syntactic analysis is completed.

Top-down parsing consists of four main components for its execution:

- **Input buffer:** The terminal sequence delivered by the Lexer (Table 2) followed by `$`.
- **Stack:** Initially contains `$ Program` (top to the right).
- **Table $M[A,a]$:** The parsing table (Table 13).
- **Output:** The sequence of production rules applied, forming the parse tree.

Lexer tokens

From the previous assignment, 'Lexical Analyzer', a token classification was defined for string processing. The Lexer delivers each token as a terminal symbol of a grammar; consequently, the Parser does not process individual characters, only terminal sequences. The token classification is shown in the table below.

Lexer token (TokenType)	Grammar terminal
INT, FLOAT, CHAR	`int, float, char`
ID	`id`
INT_CONST	`num`
FLOAT_CONST	`real`
CHAR_CONST	`chr`
ASSIGN	`=`
PLUS, MINUS, STAR, SLASH, PERCENT	`+ - * /%`
LPAREN, RPAREN, SEMI	`( ) ;`
STRING	`Str`, outside the grammar
Any other (return, ==, true...)	its own lexeme (not in the grammar)

Table 2. Lexer token and its corresponding terminal in the grammar.

As an example of using tokens as terminal symbols in the grammar, the Lexer reads `int x = 10;` and delivers five tokens. Each token is translated according to the table, with the end-of-input marker $ appended at the end.

Lexeme	Lexer token	Terminal	Line
`int`	INT	`int`	1
`x`	ID	`id`	1
`=`	ASSIGN	`=`	1
`10`	INT_CONST	`num`	1
`;`	SEMI	`;`	1
(end)	—	`$`	1

Table 3. Token example for `int x = 10;`

The parser input is then `int id = num ; $`. Each input symbol retains its corresponding terminal, lexeme, and line from the original token. The parser makes decisions based on the terminal and uses the remaining data for attributes and diagnostics, as error messages require them (e.g., `missing ';' after '10', error in line 1`).

The specifications of this project require that the productions match the input tokens; therefore, no terminals were created beyond those already produced by the Lexer: each terminal of the grammar corresponds to a `TokenType` from the Lexer's 45-type catalog, and Table 2 gives the mapping between the `TokenType` names and the terminal names used in the grammar (e.g., `INT_CONST` is written as `num`).

`"num"`, `"real"`, and `"chr"` are separated because the SDT requires the type of each constant. This separation allows directly associating each terminal with its type. It would also be possible

to use a common terminal and query the original `TokenType`, but three terminals were chosen to simplify the semantic rules.

`"return"` arrives as the `return` terminal, and the Parser reports it as unexpected without needing a special rule: the lexer recognizes it, but it does not belong to the language of this grammar.

The lexer retains the recovery mechanism defined in the first stage and can continue to locate multiple errors. However, if it returns any lexical error, the driver bypasses the parser and semantic evaluation. The recovered tokens can be displayed for diagnostics, but they do not allow certifying the validity of the original text.

Parser and SDT

Below is the context-free grammar for programs consisting of declarations and assignments with arithmetic expressions, built from the proposed grammar of the "Lexical Analyzer" submission.

Non-terminal		**Productions**
`Decl`	$\rightarrow$	`Type id Decl'`
`Decl'`	$\rightarrow$	`;` | `= Value ;`
`Type`	$\rightarrow$	`int` | `float` | `char`
`Value`	$\rightarrow$	`num` | `id`

Table 4. Grammar G1 from the Lexer assignment.

The previous grammar is LL(1), and its terminals are tokens; however, this grammar only accepts a single declaration and a simple value, meaning it does not allow expressions such as `int y = x * 2;` nor sequences of multiple statements. Below is the generalized grammar for this project.

#	Non-terminal		Production
1	`Program`	$\rightarrow$	`Stmt StmtList`
2	`StmtList`	$\rightarrow$	`Stmt StmtList`
3	`StmtList`	$\rightarrow$	ε
4	`Stmt`	$\rightarrow$	`Decl`
5	`Stmt`	$\rightarrow$	`Assign`
6	`Decl`	$\rightarrow$	`Type id Decl'`
7	`Decl'`	$\rightarrow$	`;`
8	`Decl'`	$\rightarrow$	`= Expr ;`
9	`Assign`	$\rightarrow$	`id = Expr ;`
10	`Type`	$\rightarrow$	`int`
11	`Type`	$\rightarrow$	`float`
12	`Type`	$\rightarrow$	`char`
13	`Expr`	$\rightarrow$	`Term Expr'`
14	`Expr'`	$\rightarrow$	`+ Term Expr'`
15	`Expr'`	$\rightarrow$	`- Term Expr'`
16	`Expr'`	$\rightarrow$	ε
17	`Term`	$\rightarrow$	`Factor Term'`
18	`Term'`	$\rightarrow$	`* Factor Term'`
19	`Term'`	$\rightarrow$	`/ Factor Term'`
20	`Term'`	$\rightarrow$	`% Factor Term'`
21	`Term'`	$\rightarrow$	ε
22	`Factor`	$\rightarrow$	`( Expr )`
23	`Factor`	$\rightarrow$	`- Factor`
24	`Factor`	$\rightarrow$	`id`
25	`Factor`	$\rightarrow$	`num`
26	`Factor`	$\rightarrow$	`real`
27	`Factor`	$\rightarrow$	`chr`

Table 5. Complete grammar of the project (27 productions).

The previous grammar was constructed from the following context-free grammar without left recursion.

Non-terminal		Productions
E	$\rightarrow$	T E'
E'	$\rightarrow$	+ T E' $\mid \varepsilon$
T	$\rightarrow$	F T'
T'	$\rightarrow$	* F T' $\mid \varepsilon$
F	$\rightarrow$	id $\mid$ (E)

Table 6. Expression grammar without left recursion.

The previous grammar was proposed during the theoretical sessions of the Compilers course; the grammar was implemented using clearer names (Expr, Expr', Term, Term', Factor), and the operators -, /, and% were added, along with the constants num, real, and chr as factors. Value was replaced by Expr; thus, a declaration now accepts any expression.

To support programs containing sequence structures, the productions Program $\rightarrow$ Stmt StmtList and StmtList $\rightarrow$ Stmt StmtList $\mid \varepsilon$ were introduced. This structural design allows the parsing of one or more statements while intentionally rejecting empty inputs. Additionally, statements must be strictly terminated by semicolons (;) rather than line breaks. Furthermore, the inclusion of Stmt $\rightarrow$ Decl $\mid$ Assign alongside Assign $\rightarrow$ id = Expr ; enables the usage of variables after their initial declaration (e.g., x = y;), providing the explicit structure required for the Syntax-Directed Translation (SDT) phase to perform semantic checks.

The production Factor $\rightarrow$ - Factor was incorporated into the grammar to allow negative literal constants and expressions, such as -5 or -x. In alignment with the lexical analyzer design, constants are treated as unsigned at the lexer level—ensuring expressions like a-1 are correctly tokenized as a, -, and 1—leaving sign resolution entirely to the parser. While the lexer is capable of recognizing both positive and negative signs, this grammar version intentionally includes only the unary minus; expressions containing a unary plus (such as +1) are not recognized by the defined language rules.

Finally, the resulting context-free grammar is formally defined by the tuple $G = (N, T, P, S)$. The set of non-terminals N consists of 12 symbols: Program, StmtList, Stmt, Decl, Decl', Assign, Type, Expr, Expr', Term, Term', and Factor. The set of terminals T contains 16 symbols: int, float, char, id, num, real, chr, =, +, -, *, /,%, (,), and ;. Lastly, Program is designated as the start symbol S of the grammar.

Verify. Leftmost derivation

The following table shows the leftmost derivation process for the string `int id = num;`, always replacing the leftmost non-terminal. The number in parentheses indicates the production used.

Step	Rule	Sentential form
	—	`Program`
⇒	(1)	`Stmt StmtList`
⇒	(4)	`Decl StmtList`
⇒	(6)	`Type id Decl' StmtList`
⇒	(10)	`int id Decl' StmtList`
⇒	(8)	`int id = Expr ; StmtList`
⇒	(13)	`int id = Term Expr' ; StmtList`
⇒	(17)	`int id = Factor Term' Expr' ; StmtList`
⇒	(25)	`int id = num Term' Expr' ; StmtList`
⇒	(21)	`int id = num Expr' ; StmtList`
⇒	(16)	`int id = num ; StmtList`
⇒	(3)	`int id = num ;`

Table 7. Leftmost derivation of `int x = 10;`.

The grammar covers the practice example and enough for the SDT to have something to verify (types, declarations, assignments), without adding unrequested statements (such as `if` or `while`) that would make the design harder to defend. The language uses a single global scope. It does not include functions, blocks, arrays, function calls, strings, or booleans. Tokens recognized by the lexer for those constructs are rejected at this stage.

Verification of the Grammar for Top-Down Parsing

To use the chosen LL(1) predictive parser, the absence of left recursion, factoring, and alternative selection were reviewed. The initial reviews help prepare the grammar, but the definitive check is performed through FIRST, FOLLOW, and the absence of conflicts in the predictive table.

Unambiguous

A grammar is ambiguous if any string has two distinct parse trees. The typical source of ambiguity in expressions is a production like $E \to E + E \mid E * E$, which allows grouping $a + b * c$ in two different ways.

Here, that cannot happen: each precedence level is a separate non-terminal. Addition can only appear in `Expr'`, and multiplication only in `Term'`. For $a + b * c$, the only way to derive the $*$ is within the `Term` following the $+$, so there is only one possible tree. The same applies to statements: each one begins with a type (declaration) or with `id` (assignment) and ends with `;`.

Without left recursion

Left recursion occurs when a non-terminal can derive a sequence starting with itself ($A \Rightarrow^{+} A\alpha$). A top-down parser would enter an infinite loop: to expand A, it would have to expand A again without consuming input.

There is no direct or indirect left recursion. By following the non-terminals that can appear at the start of a production, dependencies move towards `Type`, `Assign`, or `Factor`, whose alternatives begin with terminals; no cycle is formed that would allow returning to the initial non-terminal without consuming input.

Left factoring

Left factoring is needed when two alternatives of the same non-terminal begin with the same prefix, for example $A \to ab \mid ac$: with a single lookahead token, the parser could not decide which alternative to apply. In this grammar, that situation was already resolved in the design: the declaration prefix `Type id` is shared by declarations with and without an initializer, so it was factored out into `Decl` $\to$ `Type id Decl'`, leaving the two alternatives of `Decl'` to begin with different terminals (`;` and `=`). In every other non-terminal, the alternatives begin with different terminals, as shown in Table 8, so no further factoring is required.

Condition (LL1)

For each production $A \to \alpha$, its select set is obtained as follows:

$$\text{SELECT}(A \to \alpha) = \text{FIRST}(\alpha) - \{\varepsilon\}$$

If α can derive ε, $\text{FOLLOW}(A)$ is also included.

A grammar is LL(1) if the select sets of the different alternatives for the same non-terminal are disjoint.

Non-terminal	Alternatives and their FIRST	Disjoint FIRST	If an alternative derives ε	LL(1)
`StmtList`	(2) StmtList → Stmt StmtList: { int, float, char, id } (3) StmtList → ε: { ε }	yes	FOLLOW = { $ } intersection: $\emptyset$	**holds**
`Stmt`	(4) Stmt → Decl: { int, float, char } (5) Stmt → Assign: { id }	yes	not applicable	**holds**
`Decl'`	(7) Decl' → ;: { ; } (8) Decl' → = Expr ;: { = }	yes	not applicable	**holds**
`Type`	(10) Type → int: { int } (11) Type → float: { float } (12) Type → char: { char }	yes	not applicable	**holds**
`Expr'`	(14) Expr' → + Term Expr': { + } (15) Expr' → - Term Expr': { - } (16) Expr' → ε: { ε }	yes	FOLLOW = {), ; } intersection: $\emptyset$	**holds**
`Term'`	(18) Term' → * Factor Term': { * } (19) Term' → / Factor Term': { / } (20) Term' → % Factor Term': { % } (21) Term' → ε: { ε }	yes	FOLLOW = { +, -,), ; } intersection: $\emptyset$	**holds**
`Factor`	(22) Factor → (Expr): { (} (23) Factor → - Factor: { - } (24) Factor → id: { id } (25) Factor → num: { num } (26) Factor → real: { real } (27) Factor → chr: { chr }	yes	not applicable	**holds**

Table 8. LL(1) condition check for each non-terminal.

All seven non-terminals listed in Table 8 satisfy the condition. The other five (`Program`, `Decl`, `Assign`, `Expr`, and `Term`) have a single production, so they satisfy it trivially. Therefore, the grammar is LL(1).

First

For each non-terminal A, FIRST(A) was calculated: the set of terminals with which a string derived from A can start, plus ε if A can derive the empty string. The LL(1) table uses FIRST to decide which production to apply.

1. If X is a terminal, FIRST$(X) = \{X\}$.
2. If there is a production $X \to \varepsilon$, then ε is in FIRST(X).
3. If $X \to Y_1 Y_2 \ldots Y_k$, FIRST(Y_1) without ε is added. If Y_1 can derive ε, then FIRST(Y_2) without ε is also added, and so on. If all Y_i can derive ε, ε is added.

Non-terminal	FIRST
`Factor`	{ id, num, real, chr, -, (}
`Term'`	{ *, /, %, ε }
`Term`	{ id, num, real, chr, -, (}
`Expr'`	{ +, -, ε }
`Expr`	{ id, num, real, chr, -, (}
`Type`	{ int, float, char }
`Decl'`	{ =, ; }
`Decl`	{ int, float, char }
`Assign`	{ id }
`Stmt`	{ int, float, char, id }
`StmtList`	{ int, float, char, id, ε }
`Program`	{ int, float, char, id }

Table 9. FIRST set of each non-terminal.

Follow

For each non-terminal A, FOLLOW(A) was computed: the set of terminals that can appear immediately after A in some sentential form. The LL(1) table uses FOLLOW for ε-productions: when A can derive ε, the Parser needs to know which tokens may come after A in order to decide whether to apply that production.

1. $ is in FOLLOW of the start symbol.
2. If a terminal comes after A, that terminal is added to FOLLOW(A).
3. If a non-terminal B comes after A, FIRST(B) without ε is added.
4. If A is at the end of the right-hand side, or everything that follows it can derive ε, the FOLLOW of the left-hand side is added.

Non-terminal	Rule	Contributes to FOLLOW
`Program`	1	{ $ }
`StmtList`	3	{ $ }
	3	{ $ }
`Stmt`	2 and 3	{ int, float, char, id, $ }
	2 and 3	{ int, float, char, id, $ }
`Decl`	3	{ int, float, char, id, $ }
`Decl'`	3	{ int, float, char, id, $ }
`Assign`	3	{ int, float, char, id, $ }
`Type`	1	{ id }
`Expr`	1	{ ; }
	1	{ ; }
	1	{) }
`Expr'`	3	{), ; }
	3	{), ; }
	3	{), ; }
`Term`	2 and 3	{ +, -,), ; }
	2 and 3	{ +, -,), ; }
	2 and 3	{ +, -,), ; }
`Term'`	3	{ +, -,), ; }
	3	{ +, -,), ; }
	3	{ +, -,), ; }
	3	{ +, -,), ; }
`Factor`	2 and 3	{ +, -, *, /, %,), ; }
	2 and 3	{ +, -, *, /, %,), ; }
	2 and 3	{ +, -, *, /, %,), ; }
	2 and 3	{ +, -, *, /, %,), ; }
	3	{ +, -, *, /, %,), ; }

Table 10. FOLLOW contributions for each occurrence of a non-terminal.

To compute FOLLOW(A), each occurrence of A on the right-hand side of a production is located, and the rule corresponding to what follows it is applied.

First and Follow Summary

Non-terminal	FIRST	FOLLOW
`Program`	{ int, float, char, id }	{ $ }
`StmtList`	{ int, float, char, id, ε }	{ $ }
`Stmt`	{ int, float, char, id }	{ int, float, char, id, $ }
`Decl`	{ int, float, char }	{ int, float, char, id, $ }
`Decl'`	{ =, ; }	{ int, float, char, id, $ }
`Assign`	{ id }	{ int, float, char, id, $ }
`Type`	{ int, float, char }	{ id }
`Expr`	{ id, num, real, chr, -, (}	{), ; }
`Expr'`	{ +, -, ε }	{), ; }
`Term`	{ id, num, real, chr, -, (}	{ +, -,), ; }
`Term'`	{ *, /, %, ε }	{ +, -,), ; }
`Factor`	{ id, num, real, chr, -, (}	{ +, -, *, /, %,), ; }

Table 11. FIRST and FOLLOW sets of the 12 non-terminals.

Parse table

The parsing table $M[A,a]$ was constructed: one row per non-terminal (the symbol at the top of the stack), one column per terminal (the current token), and, in each cell, the production that the parser must apply. An empty cell indicates a syntax error.

The rules for constructing the table are as follows.

For each production $A \to \alpha$:

1. **ε-productions go under FOLLOW**. If α can derive ε, the production is placed in $M[A,b]$ for each terminal b in FOLLOW(A).
2. **The remaining productions go under FIRST**. The production is placed in $M[A,a]$ for each terminal a in FIRST(α).

#	Production	FIRST(α)	$\alpha \Rightarrow^* \varepsilon$	Columns by FIRST (rule 2)	Columns by FOLLOW (rule 1)
1	`Program → Stmt StmtList`	{ int, float, char, id }	no	`int float char id`	—
2	`StmtList → Stmt StmtList`	{ int, float, char, id }	no	`int float char id`	—
3	`StmtList →` ε	{ ε }	yes	—	`$`
4	`Stmt → Decl`	{ int, float, char }	no	`int float char`	—
5	`Stmt → Assign`	{ id }	no	`id`	—
6	`Decl → Type id Decl'`	{ int, float, char }	no	`int float char`	—
7	`Decl' → ;`	{ ; }	no	`;`	—
8	`Decl' → = Expr ;`	{ = }	no	`=`	—
9	`Assign → id = Expr ;`	{ id }	no	`id`	—
10	`Type → int`	{ int }	no	`int`	—
11	`Type → float`	{ float }	no	`float`	—
12	`Type → char`	{ char }	no	`char`	—
13	`Expr → Term Expr'`	{ id, num, real, chr, -, (}	no	`id num real chr - (`	—
14	`Expr' → + Term Expr'`	{ + }	no	`+`	—
15	`Expr' → - Term Expr'`	{ - }	no	`-`	—
16	`Expr' →` ε	{ ε }	yes	—	`) ;`
17	`Term → Factor Term'`	{ id, num, real, chr, -, (}	no	`id num real chr - (`	—
18	`Term' → * Factor Term'`	{ * }	no	`*`	—
19	`Term' → / Factor Term'`	{ / }	no	`/`	—
20	`Term' →% Factor Term'`	{ % }	no	`%`	—
21	`Term' →` ε	{ ε }	yes	—	`+ - ) ;`
22	`Factor → ( Expr )`	{ (}	no	`(`	—
23	`Factor → - Factor`	{ - }	no	`-`	—
24	`Factor → id`	{ id }	no	`id`	—
25	`Factor → num`	{ num }	no	`num`	—
26	`Factor → real`	{ real }	no	`real`	—
27	`Factor → chr`	{ chr }	no	`chr`	—

Table 12. Placement of each of the 27 productions in the LL(1) table.

Parse table complete

M[A, a]	int	float	char	id	num	real	chr	=	+	-	*	/	%	(	)	;	$
Program	1	1	1	1													
StmtList	2	2	2	2													**3**
Stmt	4	4	4	5													
Decl	6	6	6														
Decl'								8								7	
Assign				9													
Type	10	11	12														
Expr				13	13	13	13			13				13			
Expr'									14	15					**16**	**16**	
Term				17	17	17	17			17				17			
Term'									**21**	**21**	18	19	20		**21**	**21**	
Factor				24	25	26	27			23				22			

Table 13. LL(1) parsing table M[A, a].

- **Blue cells:** Productions placed by FIRST.
- **Yellow cells:** ε-productions placed by FOLLOW.
- **Empty cells:** Syntax error.

The table has 51 occupied cells and none of them contains two different productions. Therefore, the grammar is LL(1) and, as a consequence, it is not ambiguous. The token `-` is included in two distinct rows: on one hand, in `Expr'` it represents subtraction (production 15), and in `Factor` it represents unary minus (production 23). They do not conflict because the non-terminal at the top of the stack indicates the current context. The occupied columns in a row show the valid options for that non-terminal. However, some of them may correspond to contexts different from the current one. For instance, `)` appears due to parenthesized expressions, but it is not valid after `10` in `int x = 10);`.

LL(1) Tests

To verify the operation of the LL(1) Parser, three inputs were executed: the string proposed in the project specifications, an error due to an empty cell, and a missing semicolon (;). These traces specify the expected behavior.

Input: `int x = 10;`

Step	Stack (top on the right)	Input	Action
1	`$ Program`	`int id = num ; $`	`Program` → `Stmt StmtList`
2	`$ StmtList Stmt`	`int id = num ; $`	`Stmt` → `Decl`
3	`$ StmtList Decl`	`int id = num ; $`	`Decl` → `Type id Decl'`
4	`$ StmtList Decl' id Type`	`int id = num ; $`	`Type` → `int`
5	`$ StmtList Decl' id int`	`int id = num ; $`	match `int`
6	`$ StmtList Decl' id`	`id = num ; $`	match `id`
7	`$ StmtList Decl'`	`= num ; $`	`Decl'` → `= Expr ;`
8	`$ StmtList ; Expr =`	`= num ; $`	match `=`
9	`$ StmtList ; Expr`	`num ; $`	`Expr` → `Term Expr'`
10	`$ StmtList ; Expr' Term`	`num ; $`	`Term` → `Factor Term'`
11	`$ StmtList ; Expr' Term' Factor`	`num ; $`	`Factor` → `num`
12	`$ StmtList ; Expr' Term' num`	`num ; $`	match `num`
13	`$ StmtList ; Expr' Term'`	`; $`	`Term'` → ε
14	`$ StmtList ; Expr'`	`; $`	`Expr'` → ε
15	`$ StmtList ;`	`; $`	match `;`
16	`$ StmtList`	`$`	`StmtList` → ε
17	`$`	`$`	accept

Table 14. Parser trace for the valid input `int x = 10;`.

- **Steps 1 to 4.** The parser expands from `Program` down to `Type`. In each step, the current token is `int`, and the table selects the production corresponding to that column.
- **Step 3.** Upon applying `Decl` → `Type id Decl'`, three symbols are pushed onto the stack in reverse order: `Decl'`, `id`, and `Type`. `Type` remains at the top because it is the first symbol to be recognized (as seen in the stack at step 4).
- **Steps 5, 6, 8, 12, and 15.** These are *match* steps: the terminal at the top matches the current token, and both are consumed.
- **Steps 13, 14, and 16.** These are ε-productions. They are applied because the current token is in the FOLLOW set of the respective non-terminal: `;` is in FOLLOW(`Term'`) and FOLLOW(`Expr'`), and `$` is in FOLLOW(`StmtList`).
- **Step 17.** Stack `$` and input `$`: accepted. The 11 applied productions correspond to the derivation in Table 7.

Input (Error due to empty cell): `int x 10;`

Step	Stack (top on the right)	Input	Action
1	$ Program	int id num ; $	Program → Stmt StmtList
2	$ StmtList Stmt	int id num ; $	Stmt → Decl
3	$ StmtList Decl	int id num ; $	Decl → Type id Decl’
4	$ StmtList Decl’ id Type	int id num ; $	Type → int
5	$ StmtList Decl’ id int	int id num ; $	match int
6	$ StmtList Decl’ id	id num ; $	match id
7	$ StmtList Decl’	num ; $	error: unexpected '10', expected '=' or ';'

Table 15. Parser trace for the invalid input `int x 10;` (empty cell error).

In step 7, the stack top is `Decl’` and the current token is `num`. The cell $M[\texttt{Decl'}, \texttt{num}]$ is empty, so the parser halts. The row for `Decl’` only accepts = or ;, which matches the error message output. The error is detected before consuming the invalid token: everything read up to that point (`int x`) constitutes the valid prefix of a program.

Input (Error due to missing semicolon): Two-line input: `int x = 10` and `float y = 2.5;`

Step	Stack (top on the right)	Input	Action
1	$ Program	int id = num float id = real ; $	Program → Stmt StmtList
2	$ StmtList Stmt	int id = num float id = real ; $	Stmt → Decl
3	$ StmtList Decl	int id = num float id = real ; $	Decl → Type id Decl’
4	$ StmtList Decl’ id Type	int id = num float id = real ; $	Type → int
5	$ StmtList Decl’ id int	int id = num float id = real ; $	match int
6	$ StmtList Decl’ id	id = num float id = real ; $	match id
7	$ StmtList Decl’	= num float id = real ; $	Decl’ → = Expr ;
8	$ StmtList ; Expr =	= num float id = real ; $	match =
9	$ StmtList ; Expr	num float id = real ; $	Expr → Term Expr’
10	$ StmtList ; Expr’ Term	num float id = real ; $	Term → Factor Term’
11	$ StmtList ; Expr’ Term’ Factor	num float id = real ; $	Factor → num
12	$ StmtList ; Expr’ Term’ num	num float id = real ; $	match num
13	$ StmtList ; Expr’ Term’	float id = real ; $	error: missing ';' after '10'

Table 16. Parser trace for a two-line input with a missing ; at the end of line 1.

The error is detected upon encountering `float` on line 2. Inserting a missing ; after `10` on line 1 can be suggested if the stack confirms that it is the required next terminal after resolving any nullable continuations of the expression. It is not sufficient for ; to simply appear among the expected columns. For example, in `int x = (10`, a closing parenthesis) is also pending, so attributing the issue solely to a missing semicolon must be avoided. If the context is not verified, the generic unexpected token message should be retained.

Designing SDT and rules

A context-free grammar cannot express constraints such as "a variable must be declared before use" or "the assigned value must match the variable’s type." To verify these conditions, it would be necessary to remember arbitrary identifiers and establish relations across distant parts of the program, which inherently depends on context. Consequently, `int x = 3.14;` is syntactically valid (it matches `int id = real ;`, which belongs to the language) and yet remains semantically

incorrect.

A Syntax-Directed Translation (SDT) addresses the previous premise by associating attributes with grammar symbols and semantic rules with production rules. Formally, the tables present a Syntax-Directed Definition (SDD): attributes and rules associated with the productions. The proposed implementation evaluates these rules over the complete parse tree. An SDT scheme places semantic actions within the bodies of the productions.

Attributes

Attribute	Class	What it stores
`Type.type`, `Type.width`	synthesized	`int`, `float` or `char`, and its size: 4, 4 or 1 bytes
`Decl'.inh`	inherited	the declared type, coming from the sibling `Type`
`Expr.type`, `Term.type`, `Factor.type`	synthesized	the type of the subexpression
`Expr'.inh`, `Term'.inh`	inherited	the type of what has already been computed to the left
`Expr'.syn`, `Term'.syn`	synthesized	the final type of the whole chain of operations
`token.lexeme`, `token.line`	lexical	the name or constant and its line, as delivered by the Lexer

Table 17. Attributes of the SDT.

- **Synthesized attribute:** Calculated from the node's children; information flows upward.
- **Inherited attribute:** Calculated from the parent or siblings; information flows downward or to the right.

Attributes can be computed through a depth-first traversal from left to right, adhering to their dependencies. In Expr $\rightarrow$ Term Expr', Term.type is obtained first and subsequently passed as an inherited attribute to Expr'. This organization preserves the type accumulated from the left.

Type System

The language has three types: `int`, `float` and `char`. Internally, a fourth type, `error`, is used to avoid repeating messages: when a subexpression has already failed, its type is `error` and the rules above it do not report it again.

Operation	Valid if	Result type
`+ - * /`	both operands are `int` or `float`	`float` if either is `float`; otherwise `int`
`%`	both operands are `int`	`int`
unary `-`	the operand is `int` or `float`	the same as the operand
assign or initialize	same type, or an `int` to a `float` variable	—
any, with an `error` operand	—	`error`, no new message

Table 18. Typing rules for each operation.

From int to float yes, from float to int no. An `int` may be assigned to a `float` variable, and the inverse conversion is rejected as a language compatibility rule. It is not claimed that the conversion preserves every value in all numeric representations.

char does not take part in arithmetic. The language is strictly typed: a `char` is only assigned to a `char` variable. `'a' + 1` and `char d = 65;` are errors.

Declare before use, and only once. An undeclared variable has no type, so nothing can be checked about it, and a second declaration of the same name is a semantic error in the global scope. A variable is registered when its declaration ends, so in `int x = x;` the `x` on the right is not yet declared.

Symbol Table and Addresses

The Lexer already creates one symbol-table entry per identifier, with its type, size and address set to TBD. The SDT completes those fields when it processes the declaration: the type and size come from `Type` (4 bytes for `int` and `float`, 1 for `char`), and the relative address is taken from a counter `offset` that starts at 0 and advances by the size of each declaration, `offset := offset + Type.width`. A variable whose type is still TBD is not declared. The sizes of 4, 4 and 1 bytes are project conventions, and the addresses are consecutive symbolic offsets without memory alignment. If a redeclaration occurs, the original entry is kept and no new address is reserved. Each run starts with `offset` at 0 and an empty error list.

Semantic rule for each production

`arith(op, a, b)`, `neg(t)`, and `assignable(target, value)` are not functions defined in the implementation; they are shorthand notation for the typing rules in Table 18. Likewise, `lookup(id)` is notation for searching the variable in the symbol table.

#	Production	Semantic rule
1	`Program` → `Stmt StmtList`	At start, set `offset := 0` and clear the errors. Process `Stmt` before `StmtList`.
2	`StmtList` → `Stmt StmtList`	Process the first statement before continuing with the list, keeping the declarations already made.
3	`StmtList` → ε	—
4	`Stmt` → `Decl`	—
5	`Stmt` → `Assign`	—
6	`Decl` → `Type id Decl'`	Set `Decl'.inh := Type.type`. Evaluate the initializer before registering the name. If it is already declared, report a redeclaration and keep the entry. If it is new, complete its fields and advance `offset`, even if the initializer failed.
7	`Decl'` → `;`	—
8	`Decl'` → `= Expr ;`	If `Expr.type` $\neq$ `error` and not `assignable(Decl'.inh, Expr.type)`, report an incompatibility.
9	`Assign` → `id = Expr ;`	Check whether the target is declared and also evaluate the expression. Compare types only if the target is declared and `Expr.type` $\neq$ `error`.
10	`Type` → `int`	`Type.type := int`; `Type.width := 4`
11	`Type` → `float`	`Type.type := float`; `Type.width := 4`
12	`Type` → `char`	`Type.type := char`; `Type.width := 1`
13	`Expr` → `Term Expr'`	`Expr'.inh := Term.type`; `Expr.type := Expr'.syn`
14	`Expr'` → `+ Term` $\texttt{Expr'}_1$	$\texttt{Expr'}_1$`.inh := arith(+, Expr'.inh, Term.type)`; `Expr'.syn :=` $\texttt{Expr'}_1$`.syn`
15	`Expr'` → `- Term` $\texttt{Expr'}_1$	same as 14, with -
16	`Expr'` → ε	`Expr'.syn := Expr'.inh`
17	`Term` → `Factor Term'`	`Term'.inh := Factor.type`; `Term.type := Term'.syn`
18	`Term'` → `*` `Factor` $\texttt{Term'}_1$	$\texttt{Term'}_1$`.inh := arith(*, Term'.inh, Factor.type)`; `Term'.syn :=` $\texttt{Term'}_1$`.syn`
19	`Term'` → `/` `Factor` $\texttt{Term'}_1$	same as 18, with /
20	`Term'` → `%` `Factor` $\texttt{Term'}_1$	same as 18, with % (`int` only)
21	`Term'` → ε	`Term'.syn := Term'.inh`
22	`Factor` → `( Expr )`	`Factor.type := Expr.type`
23	`Factor` → `-` $\texttt{Factor}_1$	`Factor.type := neg(`$\texttt{Factor}_1$`.type)`
24	`Factor` → `id`	if `id` has no type: error "not declared" and `Factor.type := error`; otherwise, `Factor.type := lookup(id).type`
25	`Factor` → `num`	`Factor.type := int`
26	`Factor` → `real`	`Factor.type := float`
27	`Factor` → `chr`	`Factor.type := char`

Table 19. Semantic rule of each production.

The line number for each error. Each rule takes the line number from the token where it fails: the `id` in rules 6, 9, and 24; the = in rules 8 and 9 (type mismatch); and the operator in rules 14 to 20 and 23.

Evaluating the SDT

Input: `int x = 10;`

The following figure shows the annotated parse tree for the input. The attributes are computed bottom-up for the synthesized ones (`Factor.type`, `Term.type`, `Expr.type`) and top-down for the inherited ones (`Decl'.inh`, `Expr'.inh`, `Term'.inh`).

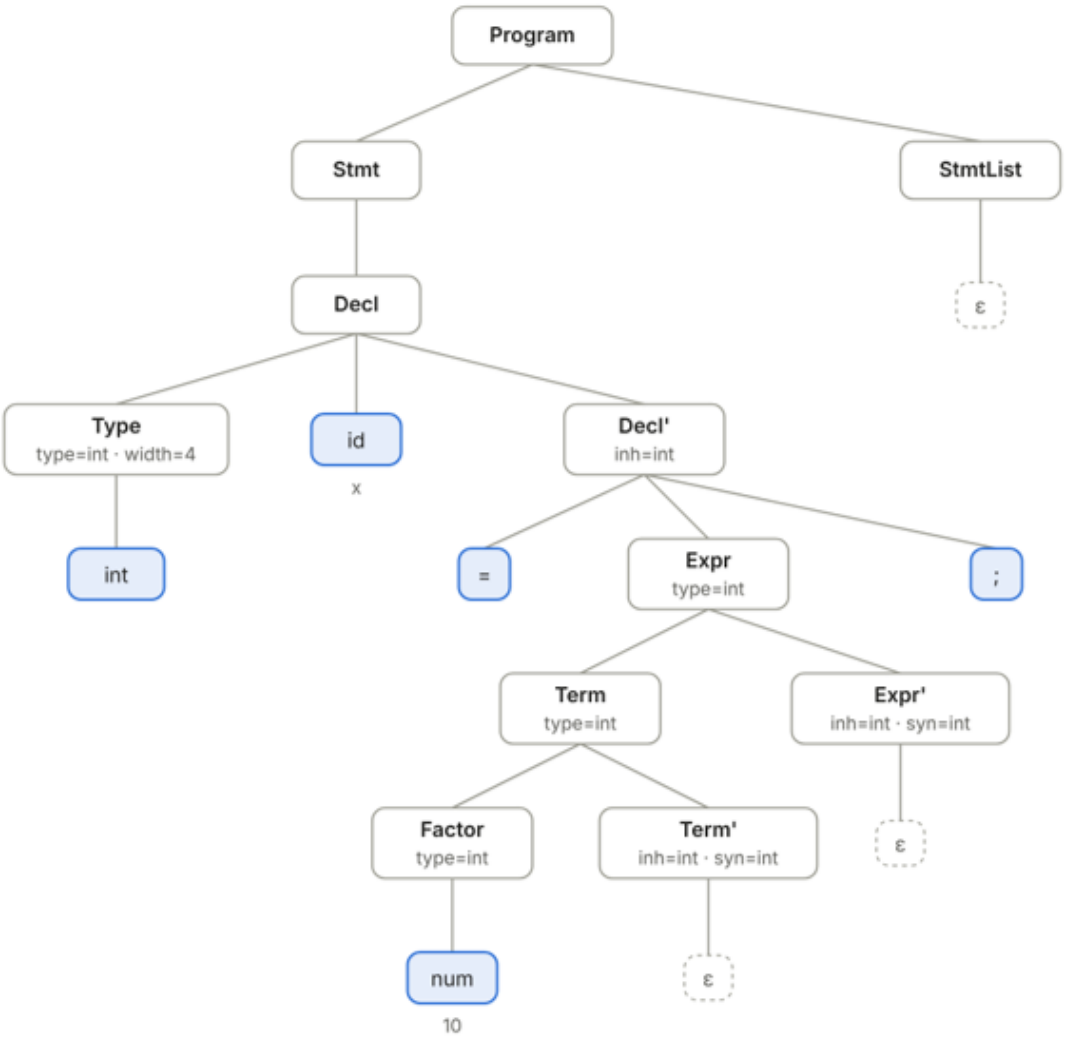


Figure 1: Annotated parse tree for `int x = 10;`.

Order	Node	Attribute	Value	Rule
1	`Type`	`type, width`	`int, 4`	10
2	`Decl'`	`inh`	`int` (comes from `Type`)	6
3	`Factor` (10)	`type`	`int`	25
4	`Term'`	`inh` $\rightarrow$ `syn`	`int` $\rightarrow$ `int`	17 and 21
5	`Term`	`type`	`int`	17
6	`Expr'`	`inh` $\rightarrow$ `syn`	`int` $\rightarrow$ `int`	13 and 16
7	`Expr`	`type`	`int`	13
8	`Decl'`	check	`assignable(int, int)` = true	8
9	`x` in the table	type, size, address	`int`, 4, 0; `offset := 4`	6

Table 20. Evaluation order of the attributes for `int x = 10;`.

The type `int` passes down from `Type` to `Decl'` as an inherited attribute; the type of `10` propagates upward from `Factor` to `Expr` as a synthesized attribute, passing through `Term'` and `Expr'` unchanged (their productions are ε). Both types meet at `Decl'`, where they are compared.

Correct parsing, incorrect SDT

Input: three-line program

```
int x = 10;
float y = x * 2.5;
x = y;
```

The first two lines are evaluated without errors, registering `x` (`int`, address 0) and `y` (`float`, address 4). Evaluation of line 3:

Order	Node	Attribute	Value	Rule
1	`id` (x)	type of the target	`lookup(x)` = `int`	9
2	`Factor` (y)	`type`	`lookup(y)` = `float`	24
3	`Term'`	`inh` $\rightarrow$ `syn`	`float` $\rightarrow$ `float`	17 and 21
4	`Term`	`type`	`float`	17
5	`Expr'`	`inh` $\rightarrow$ `syn`	`float` $\rightarrow$ `float`	13 and 16
6	`Expr`	`type`	`float`	13
7	`Assign`	check	`assignable(int, float)` = false $\rightarrow$ error at line 3	9

Table 21. Evaluation of line 3 (`x = y;`): correct parsing, but the SDT fails.

The line `x = y;` belongs to the language (it matches `id = id ;`) and is accepted by the parser. However, rule 9 detects an attempt to assign a `float` to an `int` variable—a conversion that the rules of this language do not implicitly permit. This represents the specific scenario requested by the assignment: correct parsing accompanied by an SDT/semantic error.

Test cases

The following test cases were selected from the complete test design to cover the main behaviors of the parser and the SDT. Cases whose identifier starts with **P** check the syntactic analysis (parser), and cases that start with **S** check the semantic analysis (SDT). Each case is run with the command `python parse.py -text "<input>"`, and the screenshots below show the real output of the program.

Case	Input	Expected output	What it checks
P1	`int x = 10;`	Parser: Success SDT: Verified	The example of the assignment
P4	`int x 10;`	Parser: error line 1: unexpected `'10'`, expected `'='` or `';'`	Empty cell in the $\texttt{Decl}'$ row of the parse table
S1	`float f = 10;`	Parser: Success SDT: Verified	`int` to `float` is allowed
S3	`int x = y + 1;`	SDT: error `'y'` is not declared (a single message)	Rule 24 and error propagation
S7	`char c = 'a' + 1;`	SDT: error operator needs numeric operands	`char` is not arithmetic (rule 14)

Table 22. Selected test cases for the parser and the SDT.

P1: valid input

The input `int x = 10;` is accepted by the parser and the SDT verifies it, since an `int` value is assigned to an `int` variable.

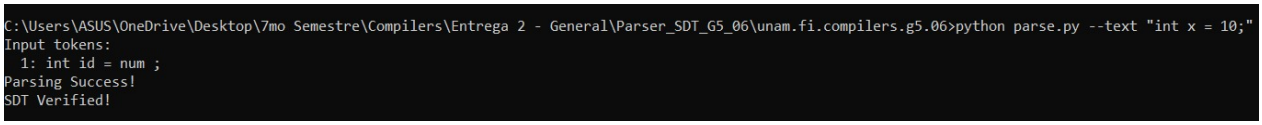


Figure 2: Execution of case P1: `int x = 10;`.

P4: parsing error

The input `int x 10;` has no = between the identifier and the number. The parse table has no entry for $\texttt{Decl}'$ with the token `num`, so the parser reports the error. The SDT stage then reports the same error, because there is no complete parse tree to evaluate.

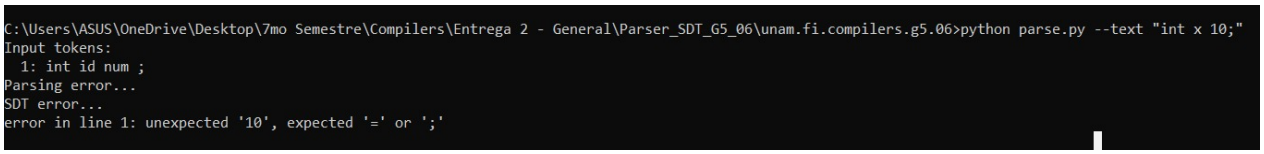


Figure 3: Execution of case P4: `int x 10;`.

S1: implicit conversion allowed

The input `float f = 10;` assigns an `int` to a `float` variable. This conversion is allowed by the type system, so both the parser and the SDT succeed.

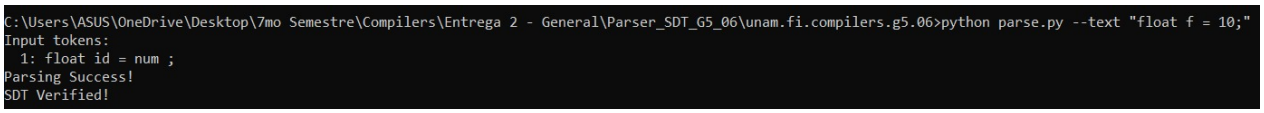


Figure 4: Execution of case S1: `float f = 10;`.

S3: undeclared variable

The input `int x = y + 1;` is syntactically correct, but y was never declared. Rule 24 reports the error once, and the error type propagates upward without producing additional messages.

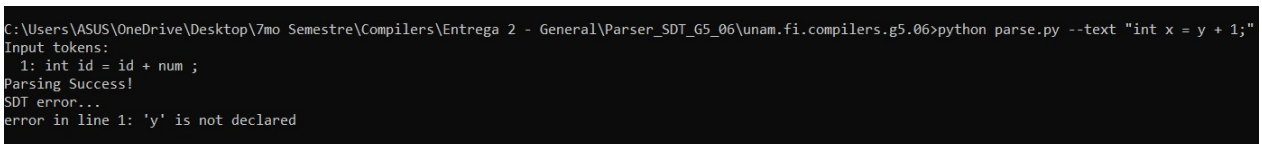


Figure 5: Execution of case S3: `int x = y + 1;`.

S7: non-numeric operand

The input `char c = 'a' + 1;` is accepted by the parser, but the SDT rejects it because the operator + cannot be applied to a `char` and an `int` (rule 14).

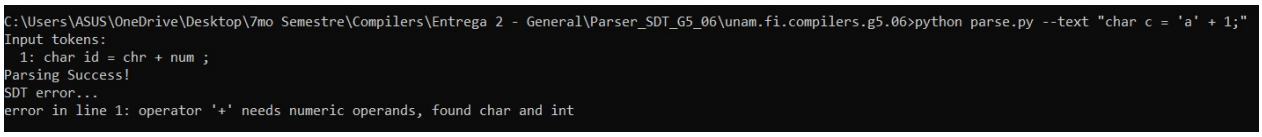


Figure 6: Execution of case S7: `char c = 'a' + 1;`.

Conclusions

In this project we designed and implemented a top-down parser with a Syntax-Directed Translation (SDT) for a small C-like language with typed declarations, assignments, and arithmetic expressions. The grammar was transformed to remove left recursion and verified to be unambiguous

and LL(1), using the FIRST and FOLLOW sets to build a parse table without conflicts. This guaranteed that the parser can decide each production with only one token of lookahead and without backtracking.

On top of the parser, the SDT adds the semantic checks that a grammar alone cannot express. Inherited attributes (such as the type that passes from `Type` to `Decl′`) and synthesized attributes (such as `Expr.type`) are evaluated over the parse tree, together with a symbol table that stores the type, size, and address of each variable. With the type system, the program rejects undeclared variables, repeated declarations, invalid operands, and incompatible assignments, while it allows the safe conversion from `int` to `float`.

The test cases confirm the separation between both stages. Inputs such as `int x 10;` fail in the parser, because of an empty cell in the parse table, while inputs such as `int x = y + 1;` and `char c = 'a' + 1;` are syntactically correct and are rejected only by the SDT. Valid inputs such as `int x = 10;` and `float f = 10;` pass both stages. This shows that a correct syntax does not imply a correct meaning, and that both analyses are needed in a compiler.

Finally, working with a formal design (grammar, tables, attributes, and rules) before programming made the implementation more direct, and the error messages with the line number make the results easy to check. As future work, the language could be extended with more statements (conditionals and loops), and the SDT could generate intermediate code instead of only verifying types.

References

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, techniques, and tools* (2nd ed.). Addison-Wesley.

Louden, K. C. (2004). *Construcción de compiladores: Principios y práctica*. Thomson.